PYTHON PROGRAMMING
10 CASE STUDIES

Including flowchart images, problem statements, objectives, assumptions, algorithms, pseudocode, Python code, test notes, and conclusions.

Flowcharts are included as images for all ten case studies. Rates and thresholds are illustrative academic assumptions.

CASE STUDY 1 – UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance, and income; rank eligible applicants and resolve ties.

2. Objectives

• Check aggregate, subject minimum, attendance, and income.

• Rank eligible applicants by aggregate, attendance, then lower income.

• Display the final ranked list.

3. Assumptions

• Minimum aggregate 85%; minimum subject mark 75%; attendance 90% or higher; annual family income at most ■2,50,000.

• Tie-break order: higher aggregate, higher attendance, then lower family income.

4. Algorithm

1. Read applicant records.

2. Calculate aggregate percentage.

3. Check all eligibility rules.

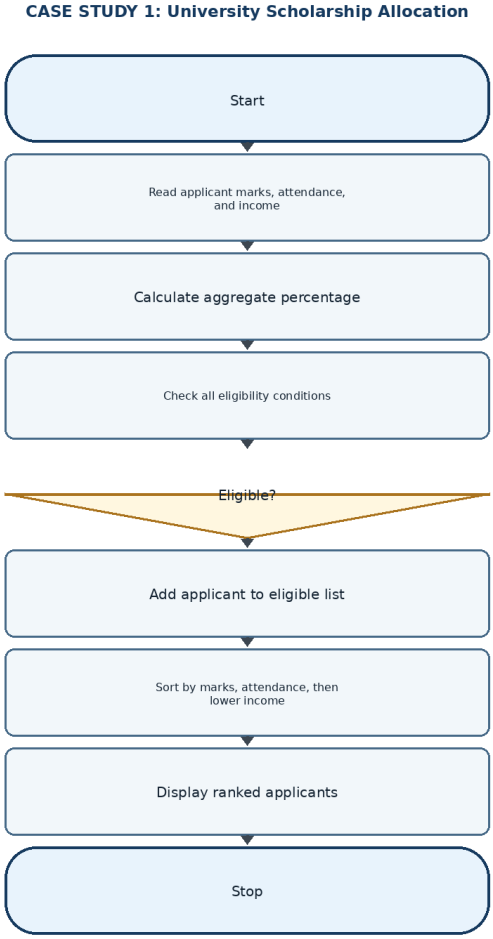
4. Sort eligible applicants by tie-break rules.

5. Display ranked applicants.

5. Pseudocode

```
START
1. Read applicant records.
2. Calculate aggregate percentage.
3. Check all eligibility rules.
4. Sort eligible applicants by tie-break rules.
5. Display ranked applicants.
STOP
```

6. Flowchart



7. Python Source Code

```
applicants = [
    ("Asha", [90, 88, 92], 95, 180000),
    ("Balu", [86, 84, 87], 92, 220000),
    ("Charan", [92, 91, 90], 90, 240000),
    ("Divya", [80, 90, 88], 96, 150000)
]
eligible = []
for name, marks, attendance, income in applicants:
    avg = sum(marks) / len(marks)
    if avg >= 85 and min(marks) >= 75 and attendance >= 90 and income <= 250000:
        eligible.append((name, avg, attendance, income))
eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))
for rank, row in enumerate(eligible, 1):
    print(rank, row)
```

8. Test Cases / Expected Behaviour

Eligible applicants are ranked by aggregate, attendance, and income.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 2 – PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

Calculate bills using consumption slabs, fixed charges, surcharges, and rebates; compare monthly bills and flag unusual increases.

2. Objectives

• Calculate slab-wise energy charges.

• Add fixed charge and surcharge; apply rebate when eligible.

• Flag a bill that rises more than 30% over the previous month.

3. Assumptions

• First 100 units: ■2/unit; next 100: ■3/unit; above 200: ■5/unit.

• Fixed charge ■75; surcharge 5% of energy charges; rebate ■50 below 100 units.

• An increase above 30% is unusual.

4. Algorithm

1. Calculate slab charges.

2. Add fixed charge and surcharge.

3. Apply rebate.

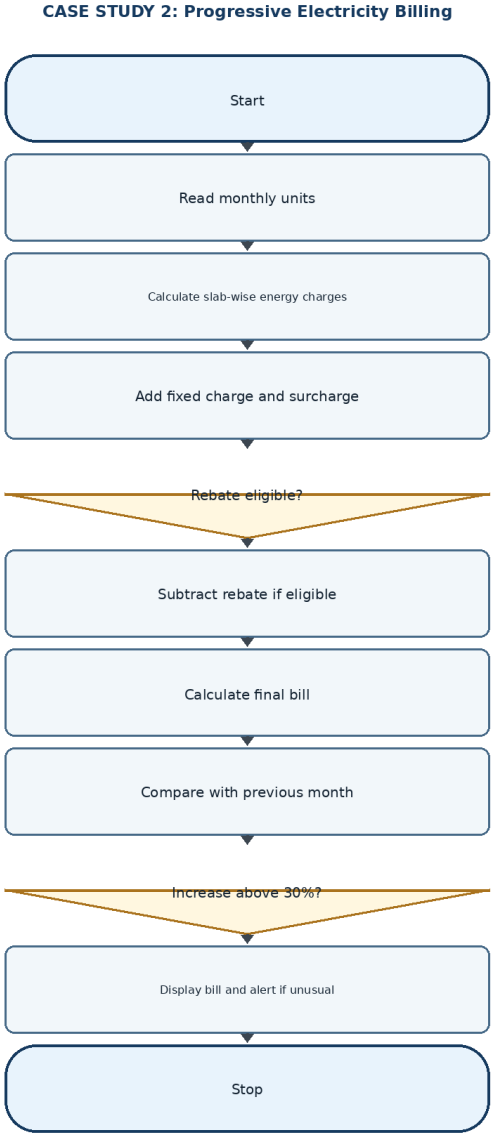
4. Compare bills month to month.

5. Display bills and alerts.

5. Pseudocode

```
START
1. Calculate slab charges.
2. Add fixed charge and surcharge.
3. Apply rebate.
4. Compare bills month to month.
5. Display bills and alerts.
STOP
```

6. Flowchart



7. Python Source Code

```
def energy_charge(units):
    return min(units, 100)*2 + max(0, min(units-100, 100))*3 + max(0, units-200)*5
monthly_units = [80, 120, 190, 280]
bills = []
for month, units in enumerate(monthly_units, 1):
    energy = energy_charge(units)
    bill = energy + 75 + energy*0.05
    if units < 100:
        bill -= 50
    bills.append(bill)
    print("Month", month, "Units:", units, "Bill: ■", round(bill, 2))
    if month > 1 and bill > bills[-2]*1.30:
        print("Unusual increase detected")
```

8. Test Cases / Expected Behaviour

Monthly bills are calculated and compared; an increase above 30% is flagged.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 3 – HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

Process consultation, diagnostic, and treatment charges; apply concessions, validate billing constraints, and classify payment status.

2. Objectives

• Add consultation, diagnostic, and treatment charges.

• Give eligible senior patients a concession.

• Calculate balance and classify payment status.

3. Assumptions

• All charges and payments must be non-negative.

• Patients aged 60 or above receive a 10% concession.

• Status is Paid, Partial Payment, or Due.

4. Algorithm

1. Read patient and billing details.

2. Validate values.

3. Calculate total and concession.

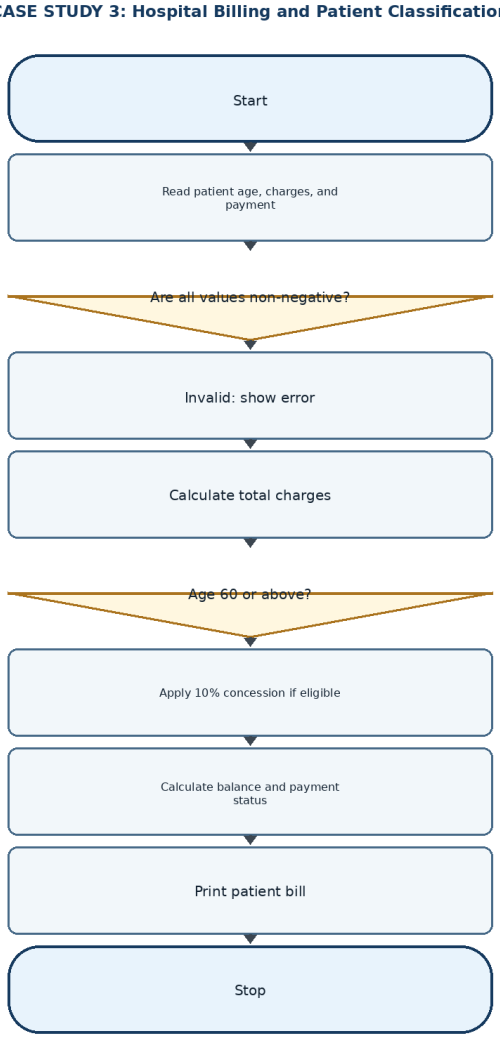
4. Calculate balance and status.

5. Print bill.

5. Pseudocode

```
START
1. Read patient and billing details.
2. Validate values.
3. Calculate total and concession.
4. Calculate balance and status.
5. Print bill.
STOP
```

6. Flowchart



7. Python Source Code

```
patients = [
    ("Asha", 65, 500, 1200, 3000, 4000),
    ("Balu", 35, 400, 800, 1800, 1000)
]
for name, age, consult, tests, treatment, paid in patients:
    if min(consult, tests, treatment, paid) < 0:
        print(name, "Invalid billing data")
        continue
    total = consult + tests + treatment
    discount = total*0.10 if age >= 60 else 0
    final_bill = total - discount
    balance = max(0, final_bill-paid)
    status = "Paid" if balance == 0 else ("Partial Payment" if paid > 0 else "Due")
    print(name, "Total:", total, "Discount:", discount, "Balance:", balance, status)
```

8. Test Cases / Expected Behaviour

Invalid values are rejected, eligible concessions are applied, and payment status is printed.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 4 – EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and print salary statements.

2. Objectives

• Calculate HRA, travel allowance, and overtime.

• Calculate gross salary.

• Apply progressive deductions and calculate net salary.

3. Assumptions

• HRA 20% of basic; travel allowance 8% of basic; overtime ■200/hour.

• Deduction: 5% of first ■25,000, 10% of next ■25,000, and 15% above ■50,000.

4. Algorithm

1. Read employee records.

2. Calculate allowances and gross salary.

3. Calculate progressive deduction.

4. Calculate net salary.

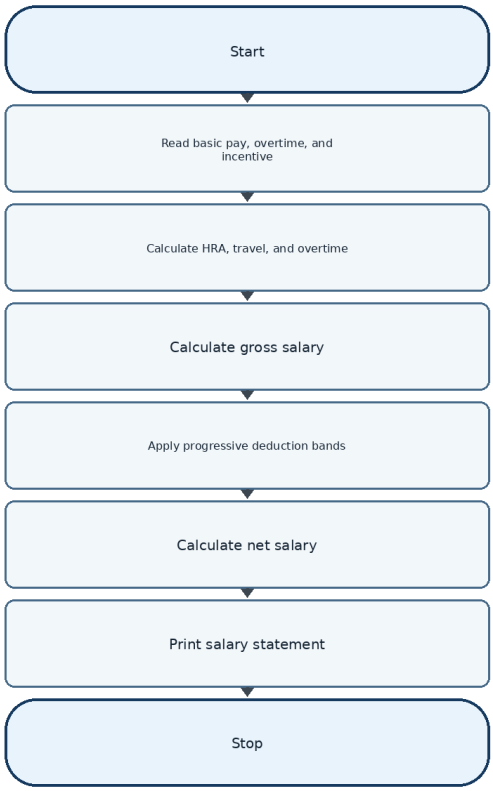
5. Print statement.

5. Pseudocode

```
START
1. Read employee records.
2. Calculate allowances and gross salary.
3. Calculate progressive deduction.
4. Calculate net salary.
5. Print statement.
STOP
```

6. Flowchart

CASE STUDY 4: Employee Payroll and Deduction Analysis



7. Python Source Code

```
employees = [("Asha", 22000, 5, 1500), ("Balu", 35000, 8, 2500), ("Charan", 50000, 10, 4000)]
for name, basic, hours, incentive in employees:
    hra, travel, overtime = basic*0.20, basic*0.08, hours*200
    gross = basic + hra + travel + overtime + incentive
    if gross <= 25000:
        deduction = gross*0.05
    elif gross <= 50000:
        deduction = 25000*0.05 + (gross-25000)*0.10
    else:
        deduction = 25000*0.05 + 25000*0.10 + (gross-50000)*0.15
    print(name, "Gross:", round(gross,2), "Deduction:", round(deduction,2), "Net:", round(gross-deduction,2))
```

8. Test Cases / Expected Behaviour

The program prints gross salary, progressive deduction, and net salary for each employee.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 5 – MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

1. Problem Statement

Compare mobile plans against usage, validity, and budget, then select the most economical eligible plan.

2. Objectives

• Calculate excess data charges for each plan.

• Check validity and budget.

• Choose the lowest-cost eligible plan.

3. Assumptions

• Usage is in GB per month.

• Minimum validity is 28 days.

• The plan must remain within the stated budget.

4. Algorithm

1. Read usage and budget.

2. Calculate each plan's total cost.

3. Filter by validity and budget.

4. Select minimum cost.

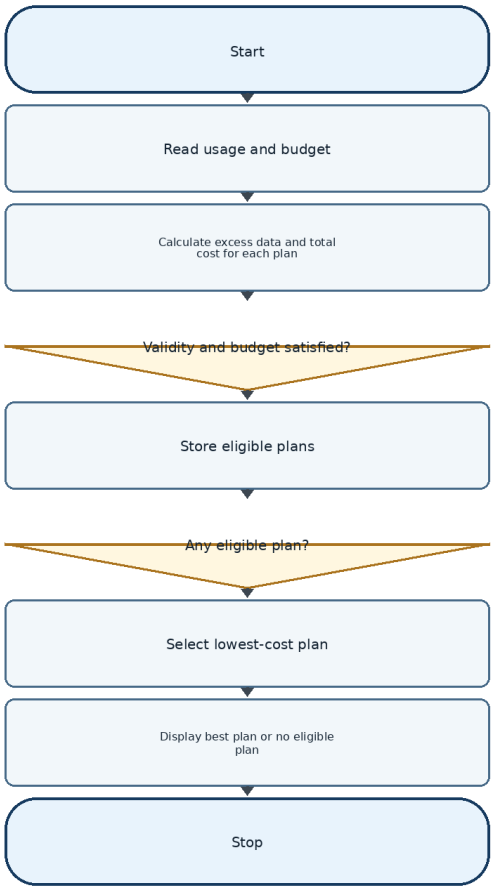
5. Display best plan.

5. Pseudocode

```
START
1. Read usage and budget.
2. Calculate each plan's total cost.
3. Filter by validity and budget.
4. Select minimum cost.
5. Display best plan.
STOP
```

6. Flowchart

CASE STUDY 5: Multi-Plan Telecommunications Cost Optimization



7. Python Source Code

```
plans = [("Basic",199,10,28,20), ("Smart",299,25,56,10), ("Max",399,40,84,5)]
needed_gb, budget = 22, 350
eligible = []
for name, price, included, validity, extra_rate in plans:
    excess = max(0, needed_gb-included)
    total = price + excess*extra_rate
    if validity >= 28 and total <= budget:
        eligible.append((total, name, validity, excess))
if eligible:
    cost, name, days, extra = min(eligible)
    print("Best plan:", name, "Cost:", cost, "Validity:", days, "Extra GB:", extra)
else:
    print("No eligible plan")
```

8. Test Cases / Expected Behaviour

The lowest-cost plan meeting usage, validity, and budget constraints is selected.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 6 – RAILWAY FARE AND RESERVATION CALCULATOR

1. Problem Statement

Calculate railway fares using distance slabs, class multipliers, age concessions, reservation fees, and cancellation penalties.

2. Objectives

• Calculate fare using distance slabs.

• Apply class multiplier and age concession.

• Add reservation fee or calculate cancellation refund.

3. Assumptions

• First 100 km: ■1.50/km; next 400 km: ■1.20/km; above 500 km: ■1/km.

• Sleeper multiplier 1.0; AC multiplier 2.0.

• Age 60+ receives 20% concession; reservation ■40; cancellation penalty 25% of fare.

4. Algorithm

1. Calculate distance-slab fare.

2. Apply class and age rules.

3. Add reservation fee.

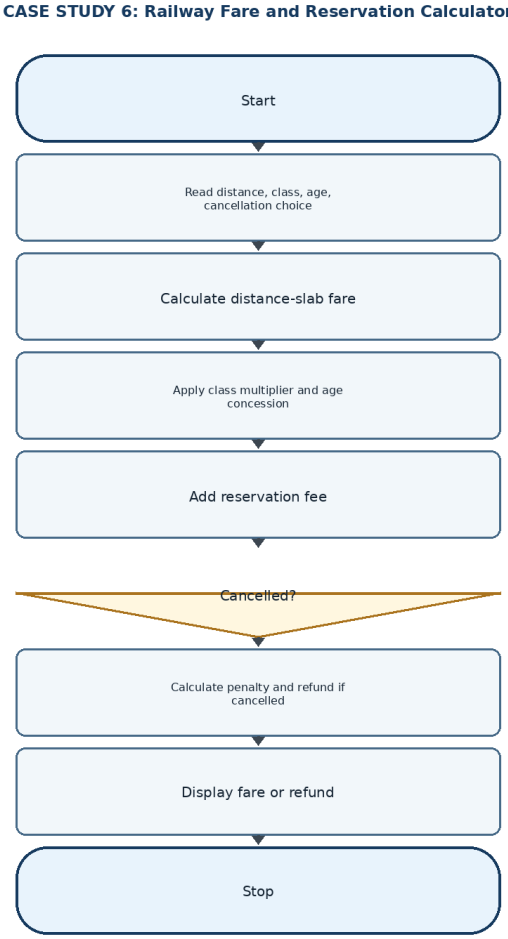
4. If cancelled, calculate penalty and refund.

5. Display result.

5. Pseudocode

```
START
1. Calculate distance-slab fare.
2. Apply class and age rules.
3. Add reservation fee.
4. If cancelled, calculate penalty and refund.
5. Display result.
STOP
```

6. Flowchart



7. Python Source Code

```
def base_fare(km):
    return min(km,100)*1.5 + max(0,min(km-100,400))*1.2 + max(0,km-500)
passengers = [("Asha",650,"Sleeper",65,False), ("Balu",300,"AC",30,True)]
for name, km, cls, age, cancelled in passengers:
    fare = base_fare(km) * {"Sleeper":1.0, "AC":2.0}[cls]
    if age >= 60:
        fare *= 0.80
    total = fare + 40
    if cancelled:
        print(name, "Refund:", round(total-fare*0.25,2))
    else:
        print(name, "Total fare:", round(total,2))
```

8. Test Cases / Expected Behaviour

Fare output includes the class, age concession, reservation fee, and cancellation refund when applicable.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 7 – RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

Calculate product totals, category-specific taxes, threshold discounts, and a maximum discount to create an itemized invoice.

2. Objectives

• Calculate item totals and subtotal.

• Apply discount if subtotal reaches threshold, capped at ■1,000.

• Calculate category taxes and final total.

3. Assumptions

• Electronics tax 18%; clothing tax 5%.

• 10% discount for subtotal ≥ ■5,000; maximum discount ■1,000.

• Discount is applied before tax.

4. Algorithm

1. Calculate subtotal.

2. Calculate capped discount.

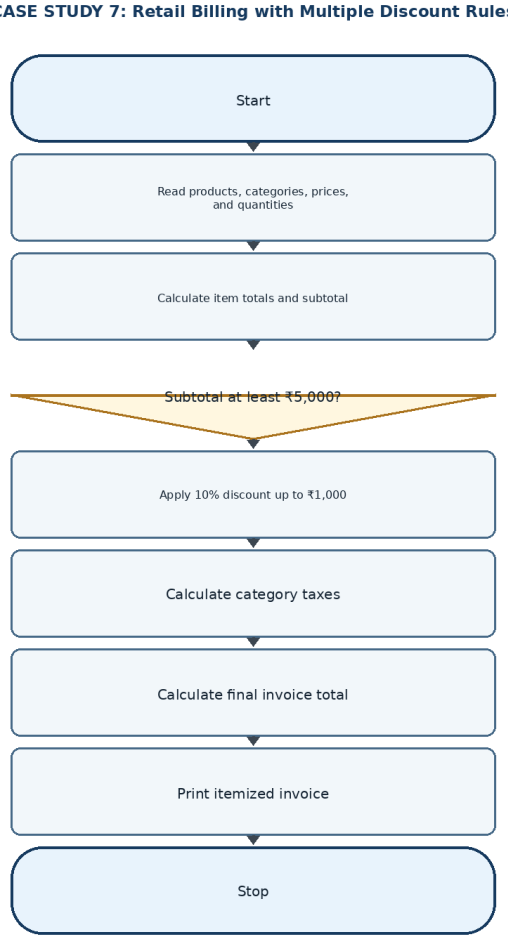
3. Apply category tax to discounted item values.

4. Print invoice.

5. Pseudocode

```
START
1. Calculate subtotal.
2. Calculate capped discount.
3. Apply category tax to discounted item values.
4. Print invoice.
STOP
```

6. Flowchart



7. Python Source Code

```
products = [("Headphones","electronics",2000,2), ("T-shirt","clothing",800,2), ("Mouse","electronics",700,1)]
subtotal = sum(price*qty for name, cat, price, qty in products)
discount = min(subtotal*0.10,1000) if subtotal >= 5000 else 0
ratio = discount/subtotal if subtotal else 0
tax_total = 0
for name, cat, price, qty in products:
    line = price*qty
    rate = 0.18 if cat == "electronics" else 0.05
    tax_total += line*(1-ratio)*rate
    print(name, "Amount:", line)
print("Subtotal:", subtotal, "Discount:", discount, "Tax:", round(tax_total,2))
print("Final total:", round(subtotal-discount+tax_total,2))
```

8. Test Cases / Expected Behaviour

The invoice displays subtotal, capped discount, category tax, and final amount.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 8 – MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. Problem Statement

Calculate material, labour, wastage, machine costs, selling revenue, and profit; check whether the profit target is achieved.

2. Objectives

• Calculate material and wastage costs.

• Add labour and machine costs.

• Calculate revenue and profit.

• Compare profit with target.

3. Assumptions

• Wastage is a percentage of material cost.

• Labour and machine costs apply to the batch.

• Target profit is ■10,000 per batch.

4. Algorithm

1. Read production details.

2. Calculate total production cost.

3. Calculate revenue and profit.

4. Check target.

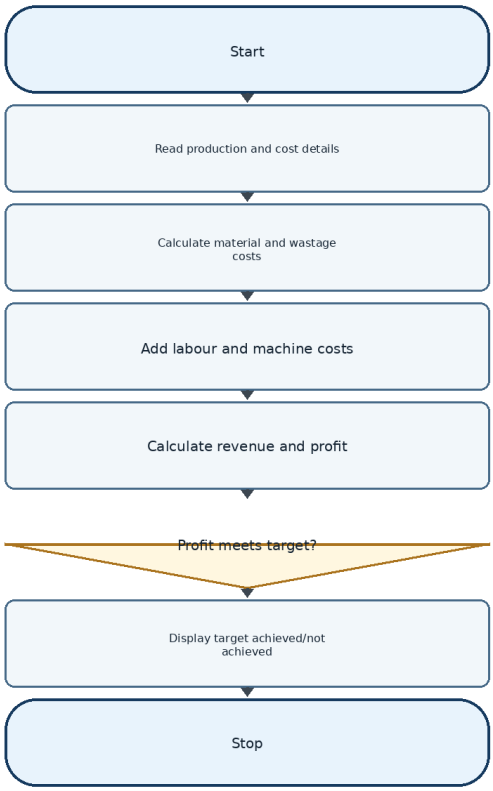
5. Display analysis.

5. Pseudocode

```
START
1. Read production details.
2. Calculate total production cost.
3. Calculate revenue and profit.
4. Check target.
5. Display analysis.
STOP
```

6. Flowchart

CASE STUDY 8: Manufacturing Cost and Profitability Analysis



7. Python Source Code

```
quantity, material_per_unit = 500, 40
wastage_percent, labour, machine = 5, 6000, 3500
selling_price, target = 75, 10000
material = quantity*material_per_unit
wastage = material*wastage_percent/100
cost = material+wastage+labour+machine
revenue = quantity*selling_price
profit = revenue-cost
print("Total cost:", cost, "Revenue:", revenue, "Profit:", profit)
print("Target profit achieved" if profit >= target else "Target profit not achieved")
```

8. Test Cases / Expected Behaviour

The program calculates total production cost and checks the resulting profit against the target.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 9 – MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. Problem Statement

Convert travel expenses using supplied exchange rates and fees, compare total spending with budget, and identify the largest expense category.

2. Objectives

• Convert each expense into INR.

• Add conversion fees.

• Compare total spending with budget.

• Identify largest expense category.

3. Assumptions

• Exchange rates are given as INR per unit of currency.

• Conversion fee is 2%.

• Budget is stated in INR.

4. Algorithm

1. Convert each expense.

2. Add fee.

3. Calculate total and remaining budget.

4. Find largest expense category.

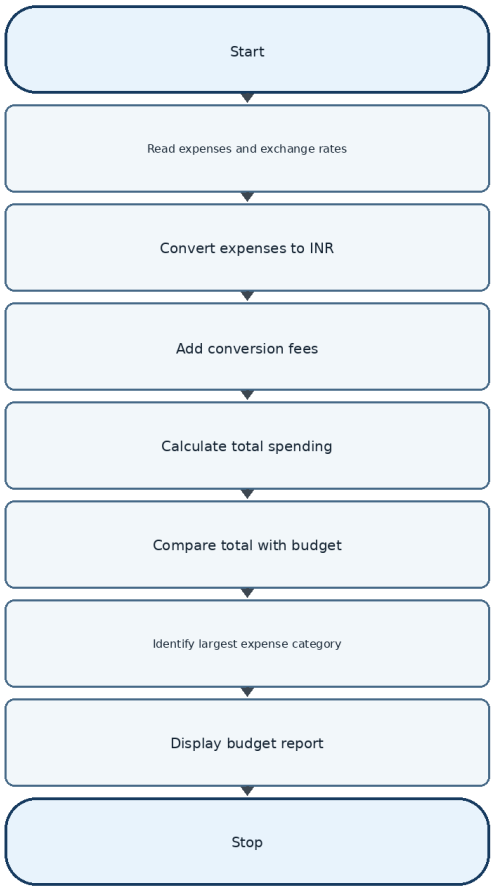
5. Display report.

5. Pseudocode

```
START
1. Convert each expense.
2. Add fee.
3. Calculate total and remaining budget.
4. Find largest expense category.
5. Display report.
STOP
```

6. Flowchart

CASE STUDY 9: Multi-Currency Travel Budget Analysis



7. Python Source Code

```
expenses = [("Accommodation",300,83.0), ("Food",120,83.0), ("Transport",90,90.0), ("Activities",150,1.0)]
budget = 50000
totals = []
for category, amount, rate in expenses:
    total = amount*rate*1.02
    totals.append((category,total))
    print(category, "INR:", round(total,2))
spent = sum(value for category,value in totals)
print("Total:", round(spent,2), "Budget:", budget, "Remaining:", round(budget-spent,2))
print("Largest category:", max(totals,key=lambda x:x[1])[0])
print("Budget exceeded" if spent > budget else "Within budget")
```

8. Test Cases / Expected Behaviour

The report includes converted category costs, fees, total spending, remaining budget, and largest expense.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.

CASE STUDY 10 – CAMPUS EVENT FINANCIAL PLANNING

1. Problem Statement

Calculate event costs, participant contributions, sponsorship shortfall, and minimum registrations needed to break even.

2. Objectives

• Calculate fixed and per-participant costs.

• Calculate total costs and income.

• Find funding shortfall.

• Calculate minimum registrations to break even.

3. Assumptions

• Venue, equipment, and transport are fixed costs.

• Catering and certificates are per-participant costs.

• Registration fee is ■500 per participant; sponsorship is fixed.

4. Algorithm

1. Add fixed costs.

2. Calculate variable cost per person.

3. Calculate expected total and income.

4. Calculate shortfall and break-even count.

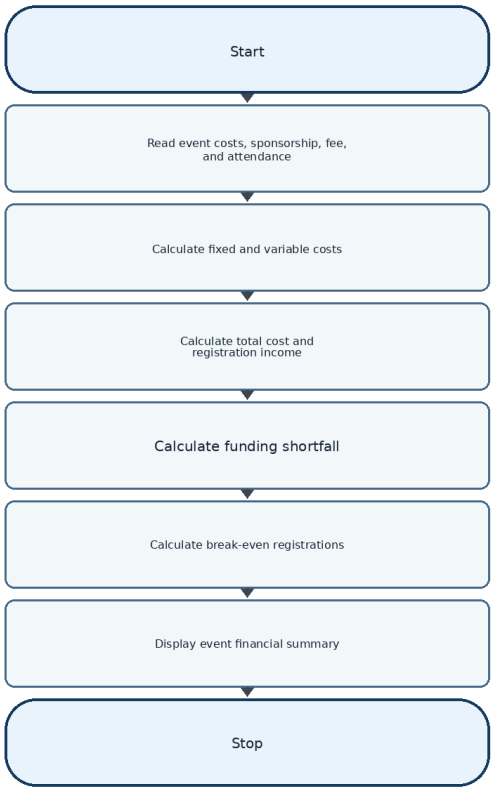
5. Display summary.

5. Pseudocode

```
START
1. Add fixed costs.
2. Calculate variable cost per person.
3. Calculate expected total and income.
4. Calculate shortfall and break-even count.
5. Display summary.
STOP
```

6. Flowchart

CASE STUDY 10: Campus Event Financial Planning



7. Python Source Code

```
venue, equipment, transport = 12000, 5000, 4000
catering, certificate = 180, 20
sponsorship, fee, people = 10000, 500, 80
fixed = venue+equipment+transport
variable = catering+certificate
total_cost = fixed+variable*people
income = fee*people
shortfall = max(0,total_cost-income-sponsorship)
contribution = fee-variable
break_even = max(0, (max(0,fixed-sponsorship)+contribution-1)//contribution)
print("Fixed cost:", fixed, "Variable/person:", variable)
print("Total cost:", total_cost, "Registration income:", income)
print("Sponsorship:", sponsorship, "Shortfall:", shortfall)
print("Break-even registrations:", break_even)
```

8. Test Cases / Expected Behaviour

The summary shows total event cost, income, sponsorship, any shortfall, and break-even registrations.

9. Conclusion

This program applies the stated rules, performs the required calculations, and displays the result for the case study.